

Android Development Cheat Sheet

Android 17 (API 37) · AGP 9.4.1 · Kotlin 2.4.20 · Compose BOM 2026.09.00 · examples use the section's running app, **TrailMate** · Full reference: <https://developer.android.com>

PLATFORM & SDK SETTINGS

compileSdk 37 (Android 17)

targetSdk 36 — Play minimum for new apps/updates since Aug 31 2026

minSdk project choice (TrailMate: 26)

targetSdk 35: edge-to-edge enforced.

targetSdk 36: predictive back on; onBackPressed() and KEYCODE_BACK no longer dispatched → onBackPressedCallback; orientation/resizability limits ignored on sw≥600dp. Studio stable: **Quail 4** (2026.1.4).

GRADLE & VERSION CATALOG

```
# gradle/libs.versions.toml
[versions]
app = "9.4.1"
kotlin = "2.4.20"
// app/build.gradle.kts
plugins {
    alias(libs.plugins.android.application)
}
dependencies {
    implementation(platform(libs.compose.bom))
}
```

AGP 9: built-in Kotlin (no kotlin.android plugin); old variant API removed → androidComponents { onVariants {} }. KSP replaces kapt.

APP COMPONENTS & INTENTS

Activity	a UI screen / entry point
Service	work without UI
Receiver	BroadcastReceiver: system/app events
Provider	ContentProvider: share data

```
// explicit
val i = Intent(this,
    DetailActivity::class.java)
startActivity(i)
// implicit
startActivity(Intent(
    Intent.ACTION_VIEW, uri))
```

Declare in AndroidManifest.xml; <intent-filter> for implicit matching; set android:exported. TrailMateApplication = app-wide init.

ACTIVITY & FRAGMENT LIFECYCLE

Create

Start

Resume

Destroy

Stop

Pause

Rotation = destroy + recreate. Fragments add onAttach, onCreateView, onCreateView, onDestroyView, onDetach; the **view** lifecycle is shorter → observe with viewLifecycleOwner, null the binding in onDestroyView.

FRAGMENTS & FRAGMENT NAVIGATION

```
supportFragmentManager
    .commit {
        setReorderingAllowed(true)
        replace(R.id.container,
            TrailListFragment())
        addToBackStack(null)
    }
```

Host in FragmentContainerView; talk via Fragment Result API or activityViewModels().

Navigation component: XML nav_graph + NavHostFragment; **Safe Args** plugin (androidx.navigation.safeargs.kotlin) generates type-checked TrailListFragmentDirections (one factory per action, passed to findNavController()). navigate() and TrailDetailFragmentArgs, read with by navArgs(). Drawer, bottom nav and tabs wire in via NavigationUI.

VIEWS ESSENTIALS

```
override fun onCreate(
    s: Bundle?) {
    super.onCreate(s)
    b = ActivityMainBinding
        .inflate(layoutInflater)
    setContentView(b.root)
    b.list.adapter = adapter
}
```

View Binding: buildFeatures { viewBinding = true }
ConstraintLayout for flat layouts;
Material Components widgets/themes. **RecyclerView** = ListAdapter + DiffUtil + ViewHolder + LayoutManager. Custom views: override onMeasure/onDraw.

COMPOSE ESSENTIALS

```
@Composable
fun TrailCard(
    trail: Trail,
    onClick: () -> Unit,
) {
    var fav by remember {
        mutableStateOf(false)
    }
    Card(onClick = onClick) {
        Text(trail.name)
        Checkbox(fav,
            { fav = it })
    }
}
```

Row/Column/Box; LazyColumn + items(trails) {} with stable keys. Modifier order matters. Hoist state. Effects: LaunchedEffect, DisposableEffect, SideEffect, derivedStateOf. Material 3 via MaterialTheme (colour scheme, typography, shapes; dynamic colour). Animation: animate*AsState, AnimatedVisibility, AnimatedContent; gestures: clickable, draggable, pointerInput.

ACCESSIBILITY

Label images: contentDescription (null if decorative). Compose Modifier.semantics { }, mergeDescendants, role, heading(), stateDescription. Touch targets ≥48dp; text in sp; check contrast. Views: importantForAccessibility, AccessibilityDelegateCompat. Test with **TalkBack**, Accessibility Scanner, Espresso AccessibilityChecks.

STATE SAVING: WHAT SURVIVES

	Conf.	Kill	Exit
ViewModel fields	✓	✗	✗
SavedStateHandle	✓	✓	✗
onSaveInstanceState, rememberSaveable	✓	✓	✗
plain remember	✗	✗	✗
Room / DataStore	✓	✓	✓

Conf. = config change, **Kill** = process death, **Exit** = user removes task. Bundle state: small, Parcelable (@Parcelize) values only; custom types via Saver. Test process death with "Don't keep activities" or adb shell am kill.

VIEWS → COMPOSE MIGRATION

```
// Compose in XML/Fragments
ComposeView(ctx).apply {
    setViewCompositionStrategy(
        strategy)
    setContent {
        TrailMateTheme {
            TrailCard(trail) { } }
    }
}
// a View inside Compose
AndroidView(factory = {
    MapView(it) })
```

In Fragments pick the ViewCompositionStrategy that disposes on the view-tree lifecycle. **Phases:** 1 theme → 2 ComposeView → 3 AndroidView → 4 RecyclerView → LazyColumn → 5 LiveData → StateFlow → 6 Nav XML → Nav Compose → Nav 3 → 7 single Activity → 8 tests → 9 cleanup. ViewModels are shared across both worlds.

ARCHITECTURE & VIEWMODEL

UI → Domain (use cases, optional) → **Data** (TrailRepository over TrailDao + TrailApi). State flows up, events flow down; single source of truth (offline-first: Room).

```
@HiltViewModel
class TrailListViewModel
@Inject constructor(
    repo: TrailRepository,
) : ViewModel() {
    val uiState = repo
        .observeTrails()
        .map(uiState::Data)
        .stateIn(viewModelScope,
            WhileSubscribed(5000),
            UiState.Loading)
}
```

viewModelScope cancels with the ViewModel; the screen collects with collectAsStateWithLifecycle(). Use cases hold reusable business logic.

NAVIGATION (COMPOSE)

```
// Nav Compose 2.10: typed
@Serializable
data object TrailList
@Serializable
data class TrailDetail(
    val trailId: String)

NavHost(nav, TrailList) {
    composable<TrailList> {
        TrailListScreen { id ->
            nav.navigate(
                TrailDetail(id)) })
    composable<TrailDetail> {
        val r: TrailDetail =
            it.toRoute()
        TrailDetailScreen(
            r.trailId)
    }
}
```

```
// Navigation 3 (1.2.0):
// routes also : NavKey;
// back.add(TrailDetail(id))
val back =
    rememberNavBackStack(
        TrailList)
NavDisplay(
    backStack = back,
    onBack = {
        back.removeLastOrNull()
    },
    entryProvider =
        entryProvider {
            entry<TrailList> { ... }
            entry<TrailDetail> {
                TrailDetailScreen(
                    it.trailId)
            }
        })
```

ADAPTIVE & EDGE-TO-EDGE

Window size classes (width COMPACT <600dp, MEDIUM <840dp, EXPANDED) from currentWindowAdaptiveInfo(). NavigationSuiteScaffold swaps bar/rail/drawer; ListDetailPaneScaffold for list-detail. Fold posture via WindowInfoTracker. enableEdgeToEdge() in onCreate; pad with WindowInsets.safeDrawing / systemBars / ime (Views: a ViewCompat insets listener).

RESOURCES & CONFIGURATION

res/values/strings.xml, values-es/, values-night/, layout-sw600dp/: qualifiers are matched against the device configuration. Read with R.string.x / getString() in Views, stringResource(), pluralStringResource(), painterResource() in Compose. Sizes in dp, text in sp; RTL via start/end; per-app language with AppCompatDelegate. setApplicationLocales().

GRAPHICS: CANVAS & OPENGLES

onDraw(Canvas) / Compose Canvas {}. **GLSurfaceView**: own surface, fastest; **GLTextureView** (a TextureView): animatable like any View.

STORAGE: DATASTORE & ROOM

```
val Context.dataStore by
    preferencesDataStore(
        "settings")
dataStore.edit {
    it[DARK_MODE] = true
}
```

```
@Entity(tableName =
    "trails")
data class TrailEntity(
    @PrimaryKey val id: String,
    val name: String)
@Dao interface TrailDao {
    @Query("SELECT * FROM " +
        "trails ORDER BY name")
    fun observeTrails():
        FlowList<TrailEntity>
    @Upsert suspend fun
        upsertAll(
            t: List<TrailEntity>)
}
```

Room 2.8.5; Room 3 (androidx.room3, KSP-only, KMP) is stable. Paging 3: PagingSource + collectAsLazyPagingItems(). Files: scoped storage, MediaStore.

NETWORKING

```
interface TrailApi {
    @GET("trails")
    suspend fun getTrails(
        @Query("page") page: Int
    ): List<TrailDto>
}
```

val api = Retrofit.Builder().baseUrl(BASE_URL).client(okHttpClient).addConverterFactory(json.asConverterFactory("application/json".toMediaType())) .build().create<TrailApi>()

Retrofit 3 + OkHttp 5 (HttpLoggingInterceptor) + kotlinx.serialization. Ktor client: HttpClient(OkHttp) + ContentNegotiation. No secrets in source.

IMAGES & CACHING

```
AsyncImage( // Coil 3
    model = trail.photoUrl,
    contentDescription =
        trail.name,
    contentScale =
        ContentScale.Crop)
```

Coil 3.6: memory cache → disk cache → network; tune a singleton ImageLoader (SingletonImageLoader.Factory) with MemoryCache/DiskCache; per request memoryCachePolicy / diskCachePolicy.

Glide 5: Glide.with(fragment).load(url).into(iv); .diskCacheStrategy(DiskCacheStrategy.RESOURCE) (ALL, DATA, NONE, AUTOMATIC = default); AppGlideModule via KSP.

DEVICE FEATURES

CameraX use cases (Preview, ImageCapture, ImageAnalysis, VideoCapture) bound to a lifecycle. Location: FusedLocationProviderClient (approximate vs precise; background asked separately). SensorManager wrapped as a Flow. **Media3** ExoPlayer + MediaSessionService. Also BLE, NFC, WebView.

DEPENDENCY INJECTION

```
@HiltAndroidApp
class TrailMateApplication
: Application()
@AndroidEntryPoint
class MainActivity :
    ComponentActivity()
// Compose
val vm: TrailListViewModel =
    hiltViewModel()
```

Hilt 2.60.1: @Module + @InstallIn(SingletonComponent::class), @Provides/@Binds: scopes @Singleton, @ViewModelScoped. **Koin 4.2:** startKoin { modules(appModule) }, koinViewModel(); runtime resolution, no codegen.

BACKGROUND WORK

```
val req =
    OneTimeWorkRequestBuilder<
        TrailSyncWorker>()
        .setConstraints(Constraints
            .Builder()
                .setRequiredNetworkType(
                    NetworkType.CONNECTED)
                .build())
        .build()
    WorkManager.getInstance(ctx)
        .enqueue(req)
```

WorkManager (CoroutineWorker, unique/periodic, @HiltWorker) for deferrable, guaranteed work. Foreground service with a declared type for user-visible ongoing work; exact alarms need permission.

NOTIFICATIONS, FCM & WIDGETS

NotificationCompat.Builder on a NotificationChannel (API 26+); runtime POST_NOTIFICATIONS (API 33+). **FCM:** extend FirebaseMessagingService, override onNewToken/onMessageReceived. **Glance** widgets: GlanceAppWidget (provideGlance → provideContent { }) + GlanceAppWidgetReceiver.

PERMISSIONS & SECURITY

```
val launcher =
    registerForActivityResult(
        RequestPermission()
    ) { granted ->
        if (granted) takePhoto()
    }
launcher.launch(Manifest
    .permission.CAMERA)
```

Dangerous permissions at runtime (API 23+), also in the manifest; Compose has a remember... launcher equivalent. Keys/secrets in **Android Keystore** (optionally with **Tink**); BiometricPrompt, Credential Manager/passkeys; Network Security Config; Play Integrity API.

TESTING

JUnit 4 / 5	unit tests (JVM)
Mockito / MockK	mocks (Mockito 5.24, mockito-kotlin)
Robolectric	4.17: framework on JVM
Espresso	instrumented View UI tests
Compose	createComposeRule

runTest + Turbine for Flows; MockWebServer for Retrofit; Room inMemoryDatabaseBuilder; @HiltAndroidTest; UI Automator across apps; work-testing for workers.

DEBUGGING & PERFORMANCE

Logcat, debugger, Layout Inspector (recomposition counts), App Inspection; StrictMode; **LeakCanary** (debugImplementation). Lint, ktlint, detekt. Profiler/Perfetto for CPU, memory, jank. **Baseline Profiles** + Macrobenchmark for startup; **R8** (isMinifyEnabled) shrinks/optimizes release builds.

BUILD, SIGN, PUBLISH & CI

```
# signed AAB for Play
./gradlew bundleRelease
```

Play App Signing: you keep the upload key, Google the app signing key. Tracks internal → closed → open → production; new personal accounts: ≥12 testers for 14 days. Native code must support 16 KB page sizes. Libraries: com.vanniktech.maven.publish 0.37.0 → Central Portal (OSSRH retired); publishToMavenCentral uploads, the publishAndRelease... variant also releases.

GitHub Actions: actions/checkout@v7, actions/setup-java@v6 (Temurin 17), gradle/actions/setup-gradle@v6, reactivecircus/android-emulator-runner@v2.

Iruueta Docs · Guides & References → Apps → Android · Generated with AI assistance; verify against developer.android.com before relying on it in production.